

PlacePrep

College Placement Command Center

High-Level Design (HLD) & Low-Level Design (LLD)

Technical Architecture Document

Author: Amit

Version: 1.0

Date: March 2026

Stack: MERN (MongoDB, Express.js, React, Node.js)

PART 1: HIGH-LEVEL DESIGN (HLD)

1. Introduction & Project Overview

PlacePrep is a full-stack college placement preparation platform designed to help students systematically prepare for campus recruitment drives. The platform provides an integrated ecosystem combining DSA practice, AI-powered mock interviews, aptitude training, resume building, and company-specific preparation resources.

1.1 Problem Statement

College students preparing for placements face fragmented resources scattered across multiple platforms. There is no single unified tool that combines coding practice, aptitude preparation, mock interviews, and resume building. PlacePrep solves this by offering a one-stop placement preparation command center.

1.2 Key Objectives

- Provide a LeetCode-style online code compiler with 200+ curated DSA problems
- Enable AI-powered mock interviews with real-time feedback and scoring
- Offer structured aptitude preparation modules (Quant, Verbal, Logical Reasoning)
- Deliver an intelligent resume builder with ATS scoring and AI suggestions
- Track progress with dashboards, streaks, leaderboards, and spaced repetition
- Provide company-specific preparation guides and past interview experiences

1.3 Target Users

User Type	Description	Key Needs
Students (Primary)	B.Tech/BCA/MCA students in pre-placement phase	DSA practice, mock interviews, aptitude prep
Placement Cells	College TPO departments	Student progress tracking, analytics
Admins	Platform administrators	Content management, user management
Alumni/Mentors	Placed students sharing experiences	Interview experience submissions

2. System Architecture Overview

PlacePrep follows a modern 3-tier web architecture pattern with a React SPA frontend, Node.js/Express REST API backend, and MongoDB as the primary database. The system is deployed on Render with cloud-managed database services.

2.1 Architecture Pattern

The application uses a Client-Server architecture with REST API communication. The frontend (React) and backend (Node.js) are deployed as separate services enabling independent scaling and CI/CD pipelines.

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	SPA with component-based UI, fast HMR dev experience
Styling	Tailwind CSS + shadcn/ui	Utility-first CSS framework with accessible components
State Management	React Context + useReducer	Global state for auth, theme, user progress
HTTP Client	Axios	API communication with interceptors for auth tokens
Backend Runtime	Node.js 18+	Non-blocking I/O, event-driven server
Backend Framework	Express.js	REST API routing, middleware pipeline
Database	MongoDB Atlas	Document-based NoSQL for flexible schemas
ODM	Mongoose 7+	Schema validation, middleware hooks, query building
Authentication	JWT + bcrypt	Stateless auth with hashed password storage
AI Integration	Google Gemini API / OpenAI	Mock interview generation, resume analysis, code hints
Code Execution	Judge0 CE API	Sandboxed multi-language code compilation and execution
File Storage	Cloudinary / Multer	Profile images, resume PDF uploads
Deployment	Render (Web Services)	Auto-deploy from GitHub, managed SSL
Version Control	Git + GitHub	Source code management, CI triggers

2.3 High-Level Architecture Diagram (Textual)

The system follows this flow:

1. Browser (React SPA) sends HTTPS requests to the API Gateway
2. API Gateway (Express.js) routes requests through authentication middleware
3. Authenticated requests reach Route Handlers which invoke Service Layer logic
4. Service Layer interacts with MongoDB via Mongoose ODM for data operations
5. External APIs (Judge0, Gemini AI, Cloudinary) are called for specialized services
6. Responses flow back through the middleware pipeline to the React frontend

3. Module Decomposition

PlacePrep is organized into 8 core modules, each responsible for a distinct domain of functionality.

Module	Description	Key Features
Auth Module	User authentication and authorization	JWT login/signup, OAuth, role-based access (RBAC)
DSA Compiler Module	Online coding environment	Multi-language compiler, test cases, solution tracking
AI Interview Module	Mock interview engine	AI question generation, real-time feedback, scoring
Aptitude Module	Quantitative & verbal prep	MCQ engine, timed tests, topic-wise analytics
Resume Builder Module	Resume creation & analysis	Templates, ATS scoring, AI content suggestions
Progress Tracker Module	Student analytics dashboard	Streaks, heatmaps, leaderboard, spaced repetition
Company Prep Module	Company-specific resources	Interview experiences, company profiles, prep guides
Admin Module	Platform management	User CRUD, content management, analytics dashboard

4. Data Flow Architecture

4.1 Authentication Flow

User submits credentials from the React login form. The backend validates credentials against bcrypt-hashed passwords in MongoDB. On success, a JWT access token (15min expiry) and a refresh token (7-day expiry) are generated. The access token is stored in memory/localStorage on the client, while the refresh token is stored as an httpOnly cookie. Subsequent API requests include the access token in the Authorization header. The auth middleware verifies the token before forwarding to route handlers.

4.2 Code Execution Flow

The user writes code in the Monaco Editor (React component) and clicks Run/Submit. The frontend sends the code, language ID, and input to the backend. The backend forwards this to the Judge0 CE API for sandboxed execution. Judge0 returns the output, execution time, and memory usage. The backend compares output against stored test cases and returns pass/fail results along with execution metrics to the frontend.

4.3 AI Interview Flow

The user selects interview type (technical/HR/behavioral) and difficulty. The backend calls the Gemini API with a structured prompt to generate contextual questions. Questions are streamed

to the user one at a time. User responses are sent back to Gemini for evaluation with scoring rubrics. The AI returns structured feedback including score (1-10), strengths, improvements, and follow-up questions. Session data is persisted to MongoDB for history and analytics.

5. Non-Functional Requirements

Requirement	Target	Strategy
Performance	API response < 200ms (p95)	MongoDB indexing, query optimization, response caching
Scalability	Support 1000+ concurrent users	Horizontal scaling on Render, connection pooling
Availability	99.5% uptime	Health checks, auto-restart, MongoDB Atlas HA
Security	OWASP Top 10 compliant	JWT, bcrypt, CORS, rate limiting, input sanitization
Maintainability	Modular codebase	MVC pattern, ESLint, Prettier, Git workflow
Code Execution Safety	Sandboxed, no server access	Judge0 containerized execution, time/memory limits

6. Deployment Architecture

PlacePrep uses a cloud-native deployment strategy on Render with the following setup:

- Frontend: Render Static Site service auto-deploying from the /client directory on GitHub push
- Backend: Render Web Service running Node.js, auto-deploying from /server directory
- Database: MongoDB Atlas M0 (free tier) cluster with automated backups and replica sets
- Environment: Secrets managed via Render Environment Variables (JWT_SECRET, DB_URI, API keys)
- CI/CD: GitHub push triggers automatic builds and deploys on Render
- DNS/SSL: Render-managed SSL certificates with custom domain support

7. Security Architecture

Threat	Mitigation	Implementation
Brute Force Login	Rate Limiting	express-rate-limit: 5 login attempts per 15 min window
XSS Attacks	Input Sanitization	DOMPurify on frontend, express-validator on backend
CSRF	SameSite Cookies	httpOnly + SameSite=Strict for refresh tokens

Threat	Mitigation	Implementation
Injection	Parameterized Queries	Mongoose schema validation, no raw queries
Data Breach	Encryption	bcrypt (12 salt rounds) for passwords, HTTPS in transit
Unauthorized Access	RBAC	JWT middleware with role checks (student/admin/mentor)

PART 2: LOW-LEVEL DESIGN (LLD)

8. Database Schema Design (MongoDB Collections)

8.1 Users Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key (auto)	Unique user identifier
name	String	required, trim	Full name of the user
email	String	required, unique, lowercase	Login email (indexed)
password	String	required, minLength: 8	bcrypt hashed password
role	String	enum: [student, admin, mentor]	RBAC role assignment
avatar	String	default: null	Cloudinary URL for profile picture
college	String	optional	College/university name
branch	String	optional	B.Tech branch (CSE, ECE, etc.)
graduationYear	Number	optional	Expected graduation year
dsaProgress	Object	embedded	{ solved: Number, streak: Number, lastActive: Date }
aptitudeProgress	Object	embedded	{ testsCompleted: Number, avgScore: Number }
resumeUrl	String	optional	Cloudinary URL of uploaded resume
refreshToken	String	optional	Hashed refresh token for auth
createdAt	Date	auto (timestamps)	Account creation timestamp
updatedAt	Date	auto (timestamps)	Last profile update timestamp

8.2 Problems Collection (DSA)

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Unique problem identifier
title	String	required, unique	Problem title
slug	String	required, unique, indexed	URL-friendly identifier
description	String	required	Problem statement in Markdown
difficulty	String	enum: [Easy, Medium, Hard]	Difficulty classification
category	String	required	Topic (Arrays, DP, Trees, etc.)

Field	Type	Constraints	Description
tags	[String]	indexed	Searchable tags array
constraints	String	required	Input/output constraints
examples	[Object]	min: 1	{ input, output, explanation }
testCases	[Object]	min: 3	{ input, expectedOutput, isHidden }
starterCode	Object	required	{ cpp, java, python, javascript }
solution	String	optional	Editorial solution (Markdown)
companies	[String]	optional	Companies that asked this problem
acceptance	Number	default: 0	Acceptance rate percentage

8.3 Submissions Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Submission identifier
userId	ObjectId	ref: Users, indexed	Who submitted
problemId	ObjectId	ref: Problems, indexed	Which problem
code	String	required	Submitted source code
language	String	required	Language used (cpp/java/python/js)
status	String	enum: [Accepted, WA, TLE, RE, CE]	Verdict from Judge0
runtime	Number	optional	Execution time in ms
memory	Number	optional	Memory usage in KB
testCasesPassed	Number	required	Count of passed test cases
totalTestCases	Number	required	Total test cases for problem
submittedAt	Date	default: Date.now	Submission timestamp

8.4 InterviewSessions Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Session identifier
userId	ObjectId	ref: Users, indexed	Interviewee
type	String	enum: [technical, hr, behavioral]	Interview category

Field	Type	Constraints	Description
difficulty	String	enum: [easy, medium, hard]	Difficulty level
questions	[Object]	required	{ question, userAnswer, aiScore, aiFeedback }
overallScore	Number	min: 0, max: 100	Aggregate performance score
duration	Number	required	Session duration in seconds
aiSummary	String	optional	AI-generated session summary
createdAt	Date	auto	When session was conducted

8.5 AptitudeTests Collection

Field	Type	Constraints	Description
_id	ObjectId	Primary Key	Test identifier
title	String	required	Test name
category	String	enum: [quant, verbal, logical]	Aptitude category
questions	[Object]	min: 5	{ question, options[], correctAnswer, explanation }
duration	Number	required	Time limit in minutes
totalMarks	Number	required	Maximum obtainable score

8.6 Database Indexes

Collection	Index	Type	Purpose
Users	{ email: 1 }	Unique	Fast login lookup
Problems	{ slug: 1 }	Unique	URL routing
Problems	{ difficulty: 1, category: 1 }	Compound	Filtered problem listing
Submissions	{ userId: 1, problemId: 1 }	Compound	User submission history
Submissions	{ userId: 1, status: 1 }	Compound	Progress tracking queries
InterviewSessions	{ userId: 1, createdAt: -1 }	Compound	Recent sessions fetch

9. REST API Design

9.1 Authentication APIs

Method	Endpoint	Auth	Description
POST	/api/auth/register	Public	Register new user with email/password
POST	/api/auth/login	Public	Login, returns JWT + refresh token
POST	/api/auth/refresh	Cookie	Refresh access token using refresh token
POST	/api/auth/logout	JWT	Invalidate refresh token, clear cookie
GET	/api/auth/me	JWT	Get current authenticated user profile

9.2 DSA / Problems APIs

Method	Endpoint	Auth	Description
GET	/api/problems	JWT	List problems with filters (difficulty, category, page)
GET	/api/problems/:slug	JWT	Get single problem details with examples
POST	/api/problems/:id/submit	JWT	Submit code solution for evaluation
POST	/api/problems/:id/run	JWT	Run code against custom input (no save)
GET	/api/problems/:id/submissions	JWT	Get user's submissions for a problem
POST	/api/problems	Admin	Create new problem (admin only)
PUT	/api/problems/:id	Admin	Update problem details (admin only)

9.3 AI Interview APIs

Method	Endpoint	Auth	Description
POST	/api/interview/start	JWT	Start new AI interview session
POST	/api/interview/:sessionId/answer	JWT	Submit answer, get AI feedback + next question
POST	/api/interview/:sessionId/end	JWT	End session, get overall score and summary
GET	/api/interview/history	JWT	Get past interview sessions with scores
GET	/api/interview/:sessionId	JWT	Get detailed session with all Q&A and feedback

9.4 Aptitude APIs

Method	Endpoint	Auth	Description
GET	/api/aptitude/tests	JWT	List available tests by category
GET	/api/aptitude/tests/:id	JWT	Get test with questions

Method	Endpoint	Auth	Description
POST	/api/aptitude/tests/:id/submit	JWT	Submit answers, get score and explanations
GET	/api/aptitude/analytics	JWT	Get topic-wise performance analytics

9.5 Resume APIs

Method	Endpoint	Auth	Description
POST	/api/resume/upload	JWT	Upload resume PDF for ATS analysis
POST	/api/resume/analyze	JWT	AI-powered resume analysis and scoring
POST	/api/resume/generate	JWT	Generate resume from structured input
GET	/api/resume/templates	JWT	Get available resume templates

9.6 Progress & Dashboard APIs

Method	Endpoint	Auth	Description
GET	/api/progress/dashboard	JWT	Aggregated stats: problems solved, streak, rank
GET	/api/progress/heatmap	JWT	Activity heatmap data (last 365 days)
GET	/api/progress/leaderboard	JWT	College-wise or global leaderboard
GET	/api/progress/analytics	JWT	Topic-wise and difficulty-wise solve analytics

10. Backend Folder Structure & Architecture

The backend follows a modular MVC (Model-View-Controller) pattern with a service layer for business logic separation:

server/

- config/ → db.js (MongoDB connection), env.js (environment variables)
- middleware/ → auth.js (JWT verification), errorHandler.js, rateLimiter.js, validator.js
- models/ → User.js, Problem.js, Submission.js, InterviewSession.js, AptitudeTest.js
- routes/ → authRoutes.js, problemRoutes.js, interviewRoutes.js, aptitudeRoutes.js, resumeRoutes.js
- controllers/ → authController.js, problemController.js, interviewController.js, etc.
- services/ → judge0Service.js (code execution), aiService.js (Gemini API), clouinaryService.js
- utils/ → generateToken.js, apiResponse.js, asyncHandler.js, constants.js
- server.js → Express app setup, middleware chain, route mounting

11. Frontend Architecture

11.1 Component Hierarchy

The React frontend uses a feature-based folder structure:

client/src/

- components/common/ → Navbar, Sidebar, Button, Modal, Loader, ErrorBoundary
- components/auth/ → LoginForm, RegisterForm, ProtectedRoute
- components/dsa/ → ProblemList, ProblemDetail, CodeEditor, TestResults, Submissions
- components/interview/ → InterviewSetup, InterviewRoom, FeedbackCard, SessionHistory
- components/aptitude/ → TestList, QuizEngine, ScoreCard, AnalyticsChart
- components/resume/ → ResumeUpload, ATSScoreCard, ResumeBuilder, TemplateSelector
- components/dashboard/ → StatsCards, HeatmapCalendar, Leaderboard, ProgressChart
- context/ → AuthContext.js, ThemeContext.js, ProgressContext.js
- hooks/ → useAuth.js, useFetch.js, useDebounce.js, useLocalStorage.js
- services/ → api.js (Axios instance), authService.js, problemService.js
- pages/ → Home, Login, Register, Dashboard, Problems, Interview, Aptitude, Resume, Admin
- utils/ → formatDate.js, constants.js, validators.js

11.2 Key Frontend Libraries

Library	Purpose
@monaco-editor/react	Code editor with syntax highlighting, autocomplete, multi-language support
react-router-dom v6	Client-side routing with nested routes and protected routes
axios	HTTP client with interceptors for auto-attaching JWT and refresh logic
react-hot-toast	Toast notifications for success/error feedback
recharts / Chart.js	Data visualization for analytics dashboards
react-markdown	Render problem descriptions and editorials from Markdown
framer-motion	Page transitions and micro-animations
tailwindcss	Utility-first CSS framework for rapid UI development

12. Key Algorithm & Logic Details

12.1 Code Submission Pipeline

7. User writes code in Monaco Editor and clicks Submit
8. Frontend sends POST /api/problems/:id/submit with { code, languageId }
9. Backend fetches problem test cases from MongoDB

10. For each test case, backend creates a Judge0 submission with { source_code, language_id, stdin, expected_output }
11. Judge0 executes code in a sandboxed Docker container (CPU: 2s, Memory: 256MB limits)
12. Backend polls Judge0 for results using submission tokens (polling interval: 1s, max: 10s)
13. Results are aggregated: if all test cases pass, status = Accepted, else WA/TLE/RE/CE
14. Submission document is created in MongoDB with verdict, runtime, and memory metrics
15. User's dsaProgress is updated: solved count, streak calculation, and last active timestamp
16. Frontend displays results with per-test-case breakdown and comparison against best submissions

12.2 Spaced Repetition Logic

PlacePrep implements a simplified SM-2 algorithm for DSA problem revision scheduling. After solving a problem, the next review interval is calculated based on the quality of the solution (time taken, attempts needed, hints used). The intervals follow a geometric progression: 1 day, 3 days, 7 days, 14 days, 30 days. Problems that took more attempts or used hints get shorter intervals. The dashboard highlights problems due for revision.

12.3 AI Interview Scoring Rubric

Each interview answer is scored by the AI on 5 dimensions: Technical Accuracy (0-25), Problem-Solving Approach (0-25), Communication Clarity (0-20), Code Quality/Pseudocode (0-15), and Edge Case Awareness (0-15). The Gemini API receives a structured prompt with the scoring rubric, and returns a JSON response with per-dimension scores and textual feedback. Overall session score is the weighted average across all questions.

13. Error Handling Strategy

13.1 Backend Error Handling

A centralized error handling middleware catches all errors. Custom ApiError class extends Error with statusCode and isOperational flags. Async controller functions are wrapped with asyncHandler to catch promise rejections. Mongoose validation errors are transformed into user-friendly messages. Unhandled rejections and uncaught exceptions are logged and trigger graceful shutdown.

13.2 Frontend Error Handling

React ErrorBoundary components wrap major page sections to prevent full-app crashes. Axios interceptors handle 401 responses by attempting token refresh before retry. Failed refreshes redirect to login. API errors display user-friendly toast notifications via react-hot-toast. Network errors show offline indicators with retry buttons.

13.3 Standard API Response Format

All API responses follow a consistent format: { success: boolean, message: string, data: object | null, error: object | null }. Success responses use 200/201 status codes. Client errors use 400/401/403/404. Server errors use 500 with generic messages (detailed errors logged server-side only).

14. Future Enhancements Roadmap

Phase	Feature	Technical Approach
Phase 2	Real-time collaborative coding	Socket.io for WebSocket-based live code sharing
Phase 2	Video mock interviews	WebRTC integration with AI speech-to-text
Phase 3	Mobile app (React Native)	Shared API, platform-specific UI components
Phase 3	ML-powered difficulty recommendations	TensorFlow.js model trained on user solve patterns
Phase 4	College admin dashboard	Multi-tenant architecture with college-level analytics
Phase 4	Placement prediction engine	Regression model using student activity data

15. Conclusion

This document outlines the complete technical architecture of PlacePrep, covering both the high-level system design and the low-level implementation details. The MERN stack provides a cohesive JavaScript ecosystem enabling rapid development, while the modular architecture ensures maintainability and scalability. The integration of AI services (Gemini API) and code execution services (Judge0) adds powerful capabilities that differentiate PlacePrep from basic CRUD applications.

The design prioritizes security (JWT + bcrypt + RBAC), performance (indexed queries + caching), and user experience (real-time feedback + interactive dashboards). The roadmap ensures the platform can evolve from an MVP to a comprehensive placement preparation ecosystem.